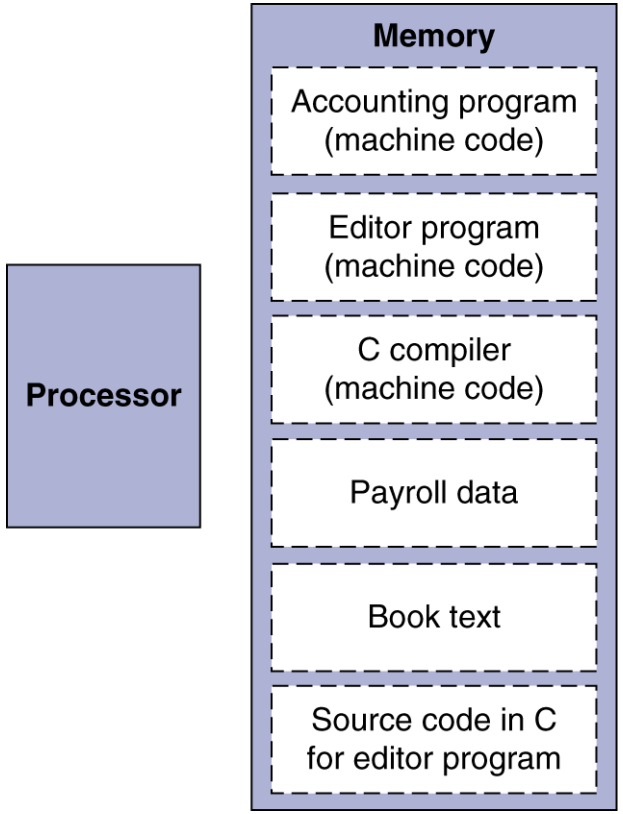
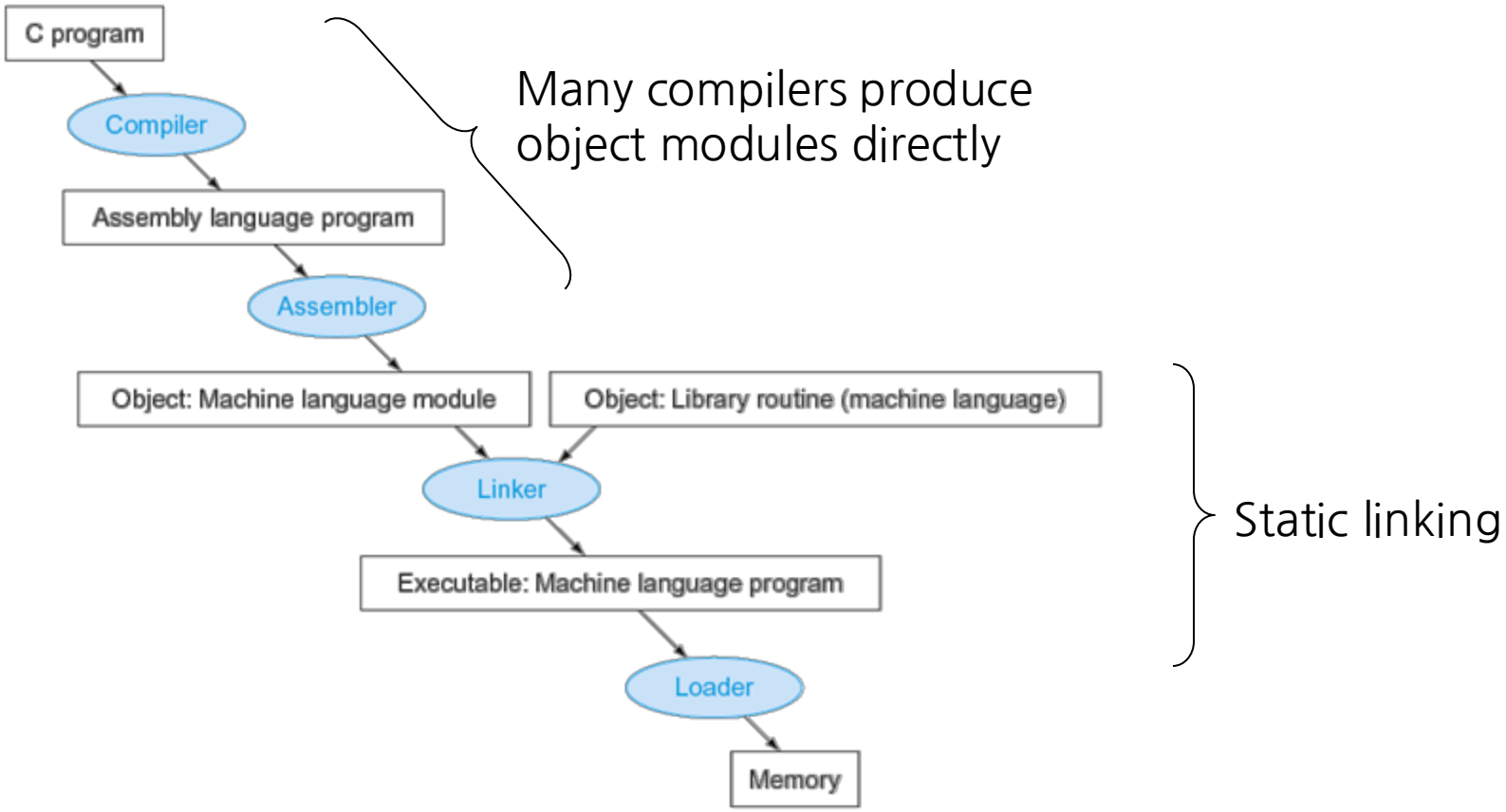
Translating and Starting a Program

Stored Program Computers



- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs

Translation and Startup



Producing an Object Module

- Assembler (or compiler) translates program into machine instructions
- Provides information for building a complete program from the pieces
  - Header: described contents of object module
  - Text segment: translated instructions
  - Static data segment: data allocated for the life of the program
  - Relocation info: for contents that depend on absolute location of loaded program
  - Symbol table: global definitions and external refs
  - Debug info: for associating with source code

Linking Object Modules

- Produces an executable image

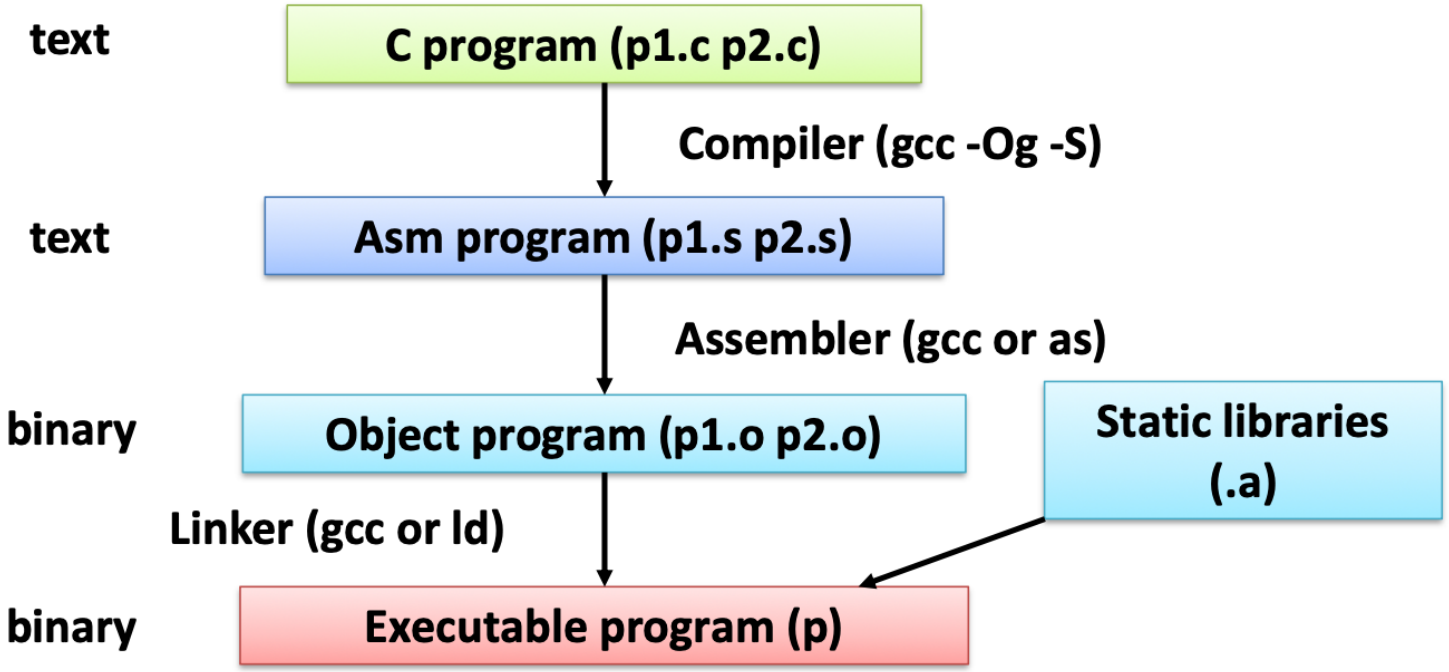
1. Merges segments
2. Resolve labels (determine their addresses)
3. Patch location-dependent and external refs

- Could leave location dependencies for fixing by a relocating loader
  - But with virtual memory, no need to do this
  - Program can be loaded into absolute location in virtual memory space

Turning C into Executable Program

```
$] riscv64-unknown-elf-gcc -Og p1.c p2.c -o p
```

- Use basic optimizations (`-Og`)
- Put resulting binary in file `p`



Example C Program

main.c

```
int buf[2] = {1, 2};

int main()
{
  swap();
  return 0;
}
```

swap.c

```
extern int buf[];

int *bufp0 = &buf[0];
static int *bufp1;

void swap()
{
  int temp;

  bufp1 = &buf[1];
  temp = *bufp0;
  *bufp0 = *bufp1;
  *bufp1 = temp;
}
```

Resolving Symbols

Global

```
int buf[2] = {1, 2};

int main()
{
  swap();
  return 0;
}
```

main.c

External

Global

External

Local

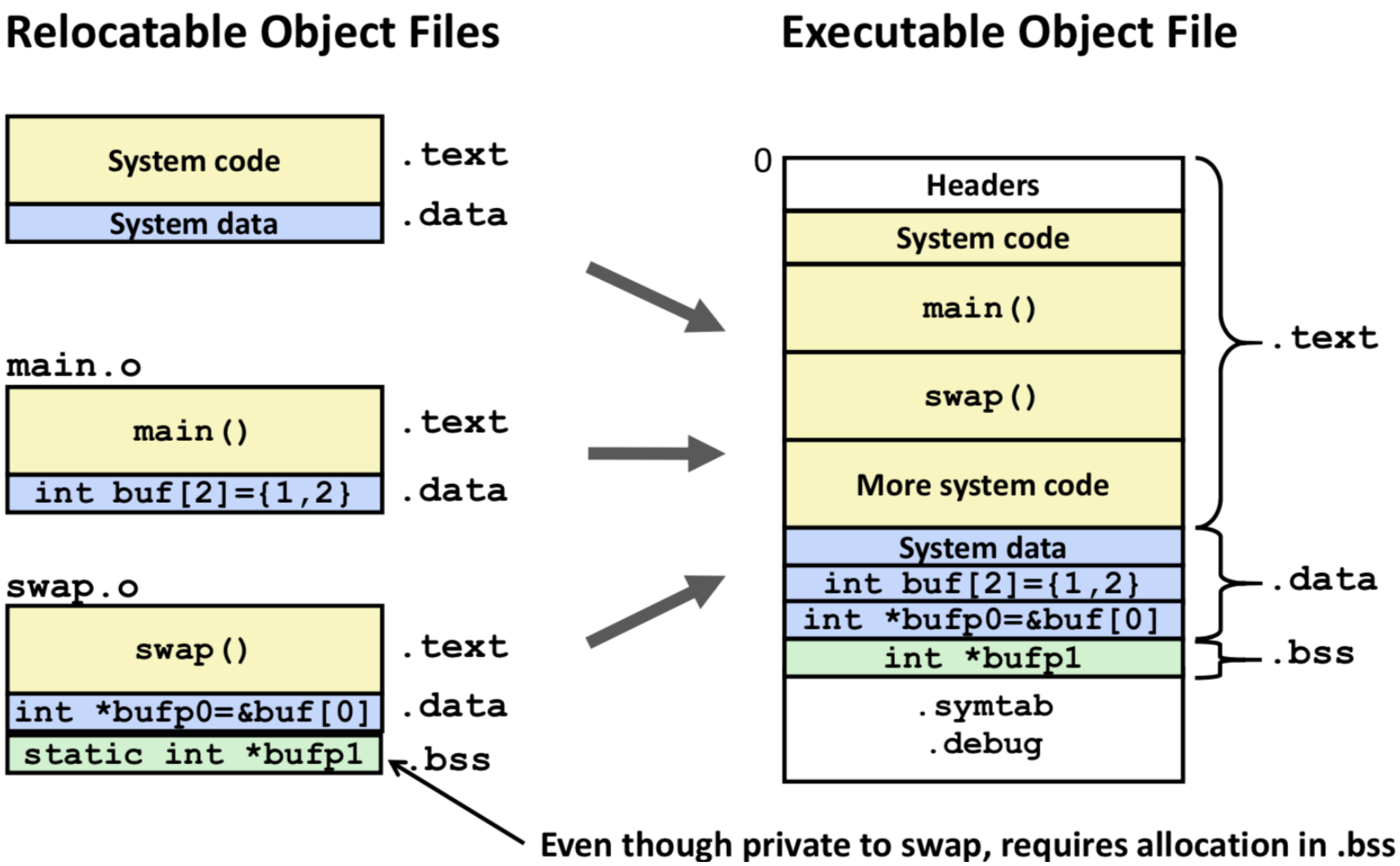
```
extern int buf[];

int *bufp0 = &buf[0];
static int *bufp1;

void swap()
{
  int temp;

  bufp1 = &buf[1];
  temp = *bufp0;
  *bufp0 = *bufp1;
  *bufp1 = temp;
}
```

swap.c

Global

Linker knows nothing of temp

Relocating Code and Data

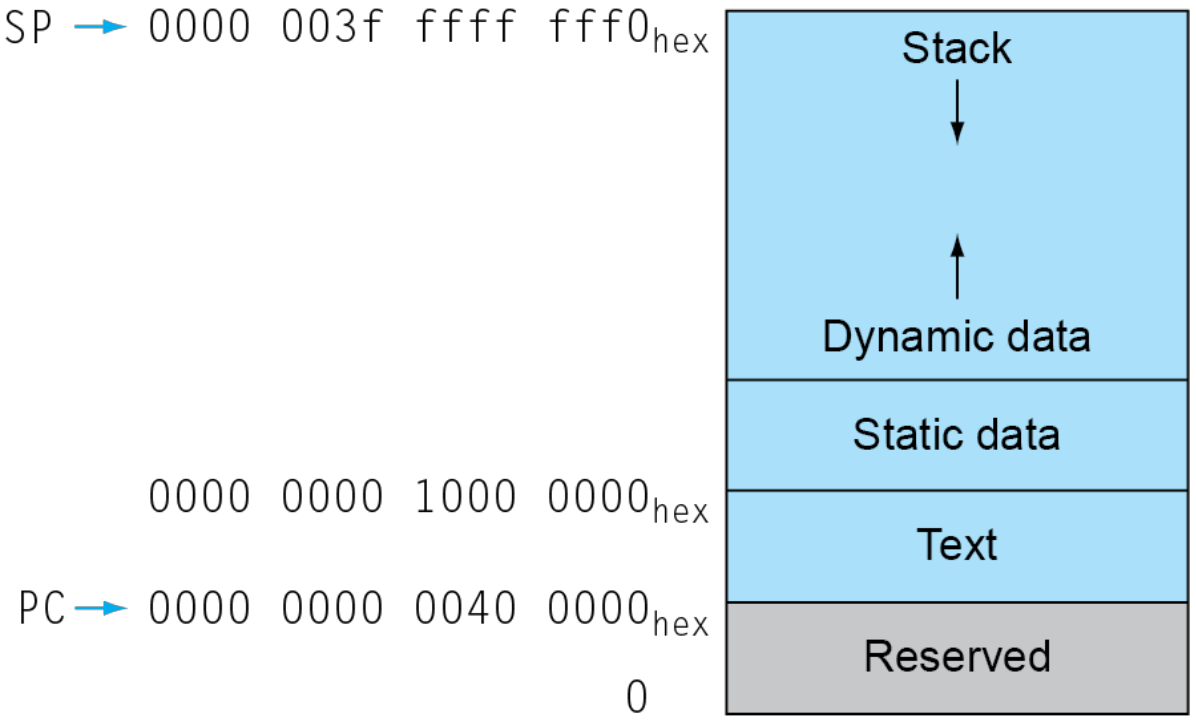


Loading a Program

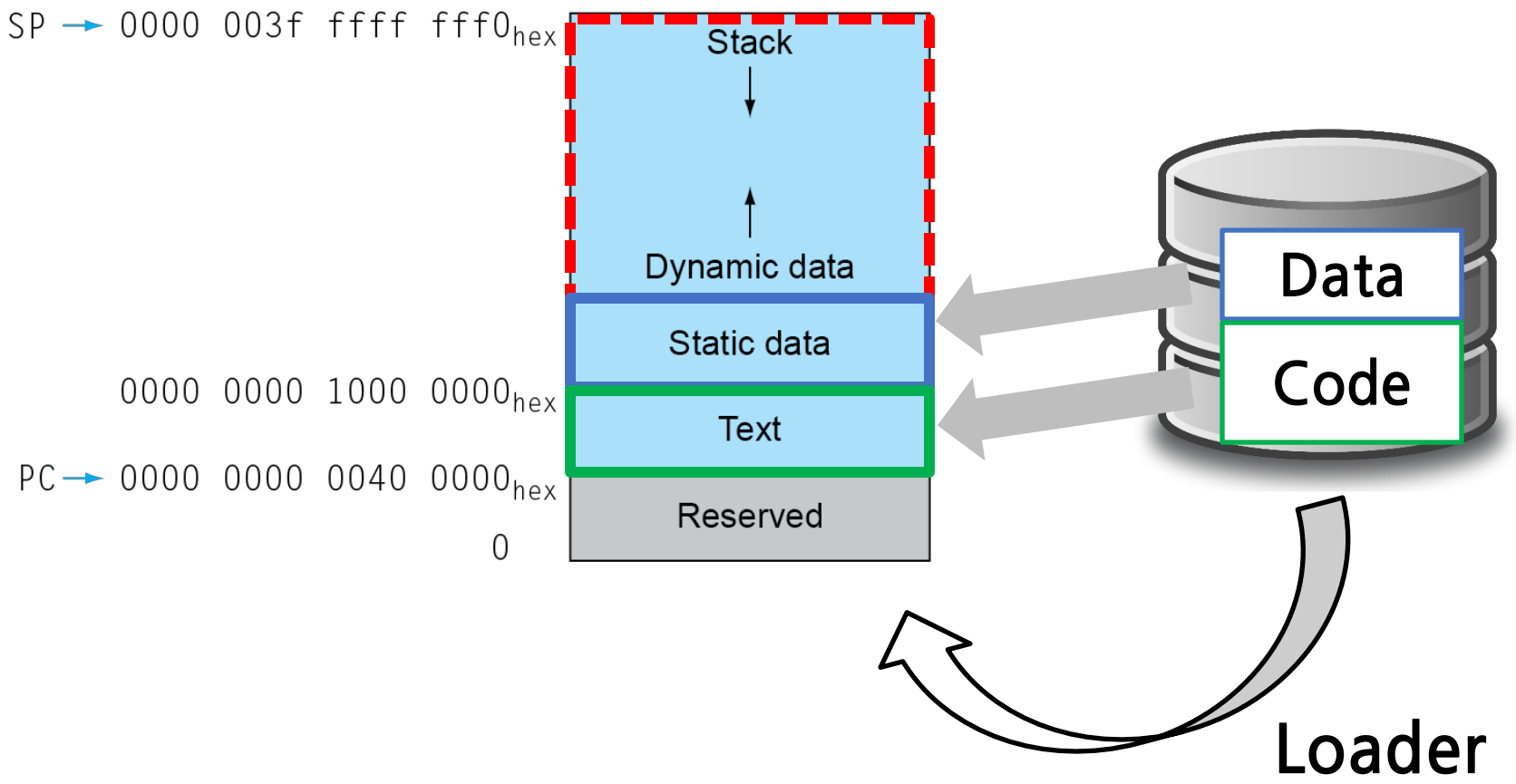
- Create a process
- Load from image file on disk into memory
  - Read header to determine segment size
  - Copy text and initialized data into memory
  - Set up stack
  - Initialize registers (including `sp, fp, gp`)
- Jump to startup routine
  - Copies arguments to `a0`, ⋯ and calls main
  - When main returns, do exit syscall

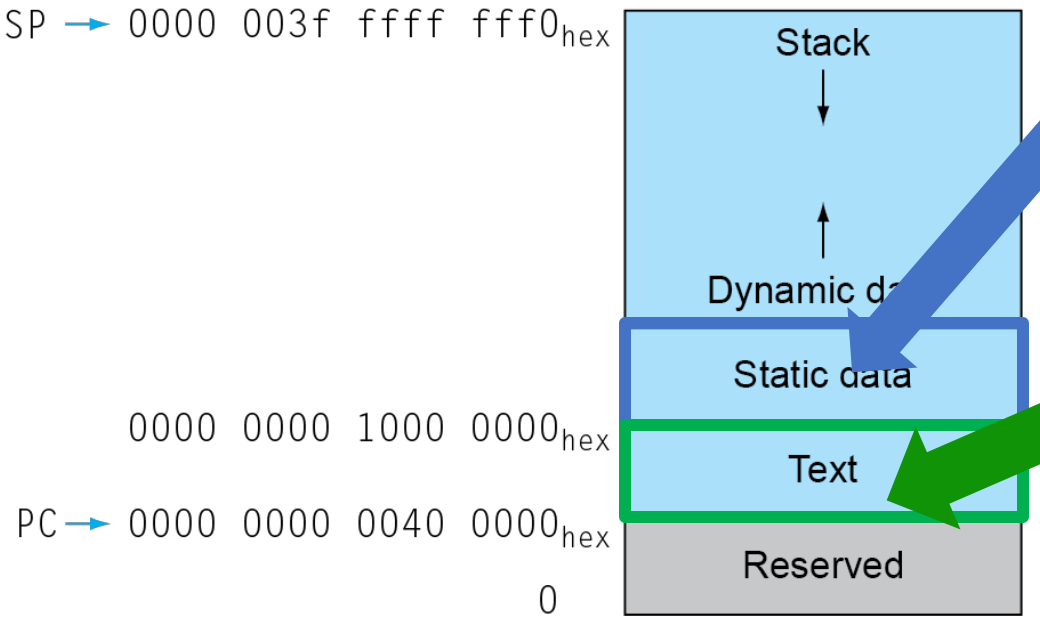
Memory Layout

- **Text**: program code
- **(Static) data**: global variables
  - e.g., static variables in C, constant arrays and strings
  - `x3` (global pointer) initialized to address allowing ±offsets into this segment
- **Heap**: dynamic data
  - e.g., `malloc` in C, `new` in Java
- **Stack**: automatic storage



From Program to Process






```
1          .data
2  array:  .word   3
3          .word   123
4          .word   4346
5  array2: .word   0x11111111
6          .text
7  main:
8          addiu   $2, $0, 1024
9          addu    $3, $2, $2
10         or  $4, $3, $2
11         sll $6, $5, 16
12         addiu   $7, $6, 9999
13         subu    $8, $7, $2
14         nor $9, $4, $3
15         ori $10, $2, 255
16         srl $11, $6, 5
17         la  $4, array2
18         and $13, $11, $5
19         andi    $14, $4, 100
20         lui $17, 100
21         addiu   $2, $0, 0xa
```

Dynamic Linking

- Only link/load library procedure when it is called
  - Requires procedure code to be relocatable
  - Avoids image bloat caused by static linking of all (transitively) referenced libraries
  - Automatically picks up new library versions

Summary

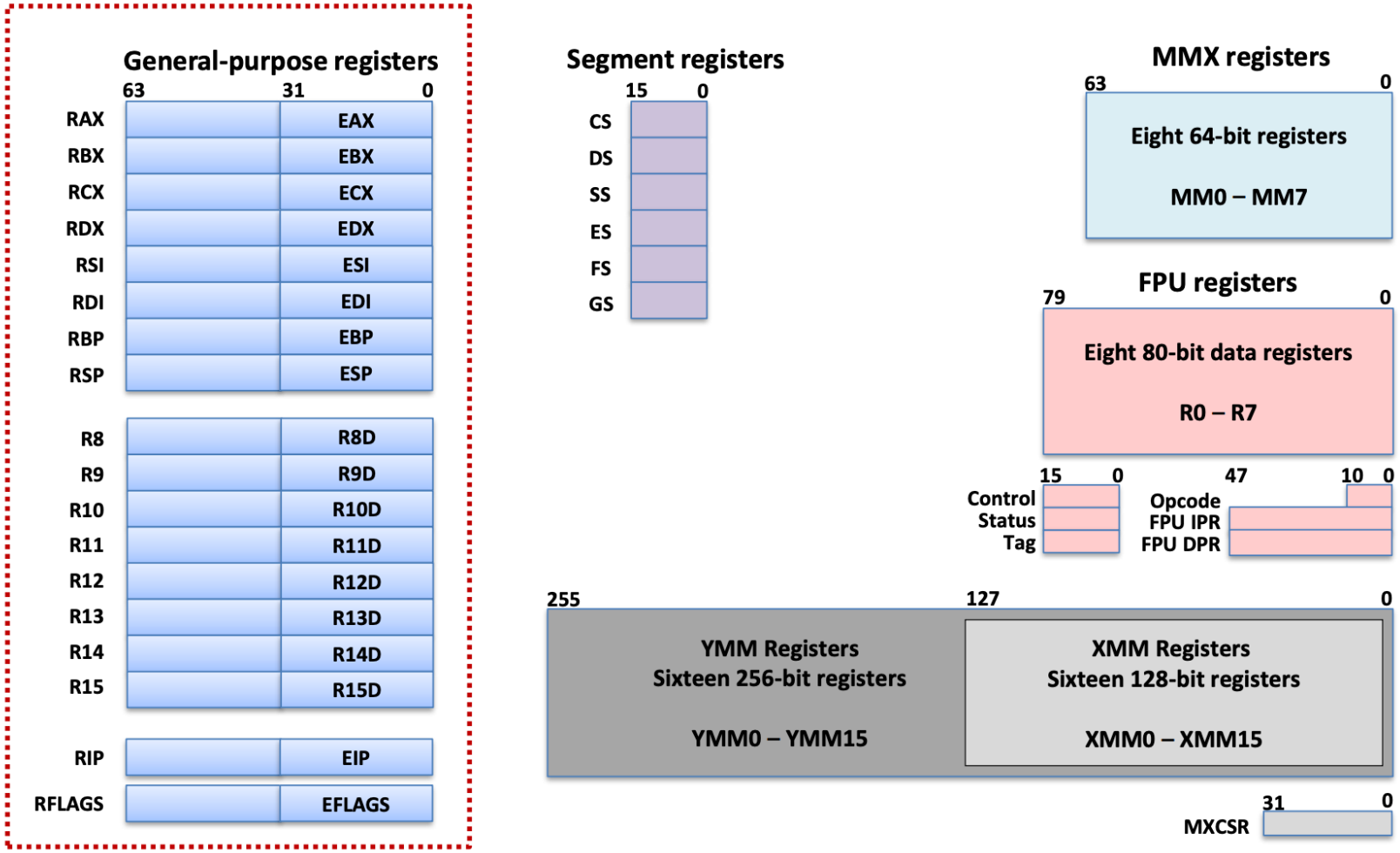
- Design principles
  1. Simplicity favors regularity
  2. Smaller is faster
  3. Make the common case fast
  4. Good design demands good compromises
- Layers of software/hardware
  - Compiler, assembler, hardware
- RISC-V: typical of RISC ISAs
  - c.f. x86

CISC vs. RISC

CISC (Complex Instruction Set Computer)

- Add instructions to perform “typical” programming tasks
  - DEC PDP-11 & VAX, IBM System/360, Motorola 68000, IA-32, Intel 64, …
- Stack-oriented instruction set
  - Use stack to pass arguments, save program counter, etc.
  - Explicit push and pop instructions
- Arithmetic instructions can access memory
  - Requires memory read and write during computation
  - Complex addressing modes
- Instructions have varying lengths
- Condition codes
  - Set as a side effect of arithmetic and logical instructions

x86 Basic Execution Environment



x86 Operands

- Two operands per instruction
- The source can be an immediate, a register, or a memory location
- The destination can be a register or a memory location

Second source operand	Source/Dest operand	Example
Register	Register	`addq %rcx, %rax`
Immediate	Register	`addq $4, $eax`
Memory	Register	`addq 0(%rcx), %rax`
Register	Memory	`addq %rax, 8(%rax)`
Immediate	Memory	`addq $4, 8(%rax)`

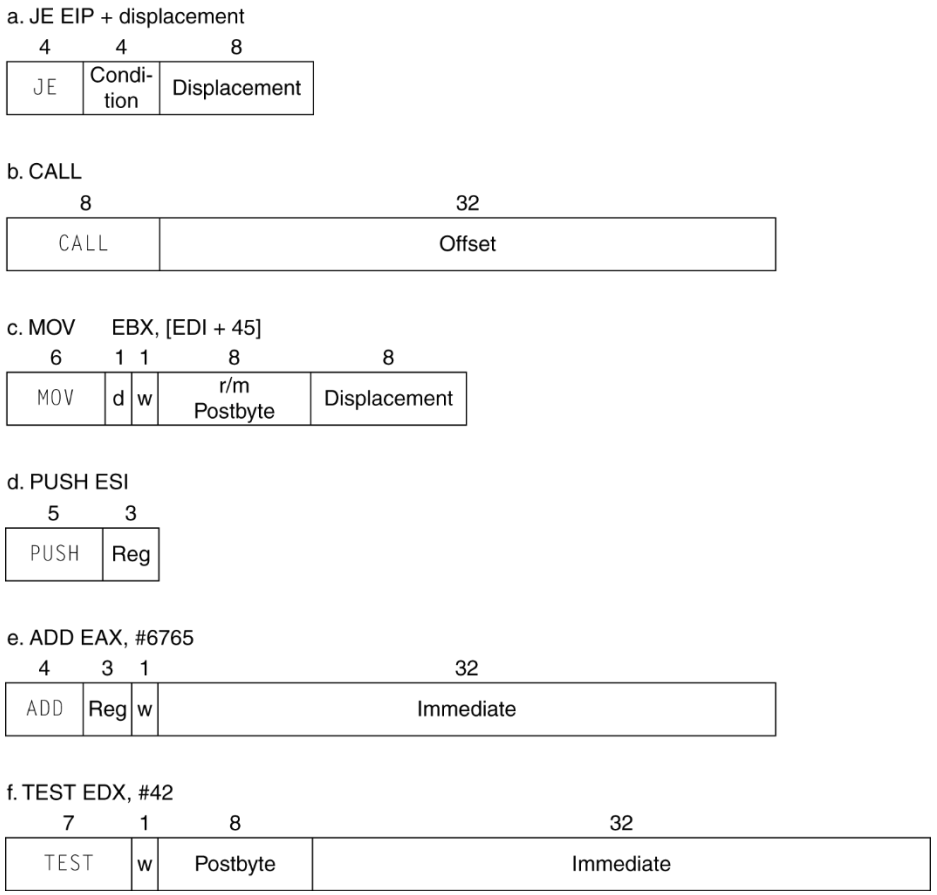
x86 Addressing Modes

- Address in register
- Address = R_{base} + displacement
- Address = $R_{base} + 2^{scale} \times R_{index}$ (scale = 0, 1, 2, or 3)
- Address = $R_{base} + 2^{scale} \times R_{index}$ + displacement

Example	Effective address
`movq %rax, 8(%rdx)`	`%rdx + 8`
`movq %rax, 16(%rdx, %rcx)`	`%rdx + %rcx + 16`
`movq %rax, (, %rcx, 4)`	`%rcx * 4`
`movq %rax, 32(%rdx, %rcx, 8)`	`%rdx + %rcx * 8 + 32`

x86 Instruction Encoding

- Variable length encoding
  - Postfix bytes specify addressing mode
  - Prefix bytes modify operation
    - Operand length, repetition, locking, …
  - The length of an instruction can be up to 17bytes
    - Up to 4 prefixes, each of which is 1 byte
    - 1 ~ 3 bytes for opcode
    - Up to 1 byte for simple addressing modes
    - Up to 1 byte for complex addressing modes
    - Up to 4 bytes for displacement
    - Up to 4 bytes for immediate data



RISC (Reduced Instruction Set Computer)

- Philosophy: fewer, simple instructions
  - Make take more to get given task done
  - Can execute them with small and fast hardware
  - Stanford MIPS, UCB RISC-V, Sun SPARC, IBM Power/PowerPC, ARM, …
- Register-oriented instruction set
  - Many more (typically 32+) registers
  - Use for arguments, return address, temporaries
- Only load and store instructions can access memory
- Each instruction has fixed size
- No condition codes
  - Test instructions return 0/1 in register

CISC vs. RISC

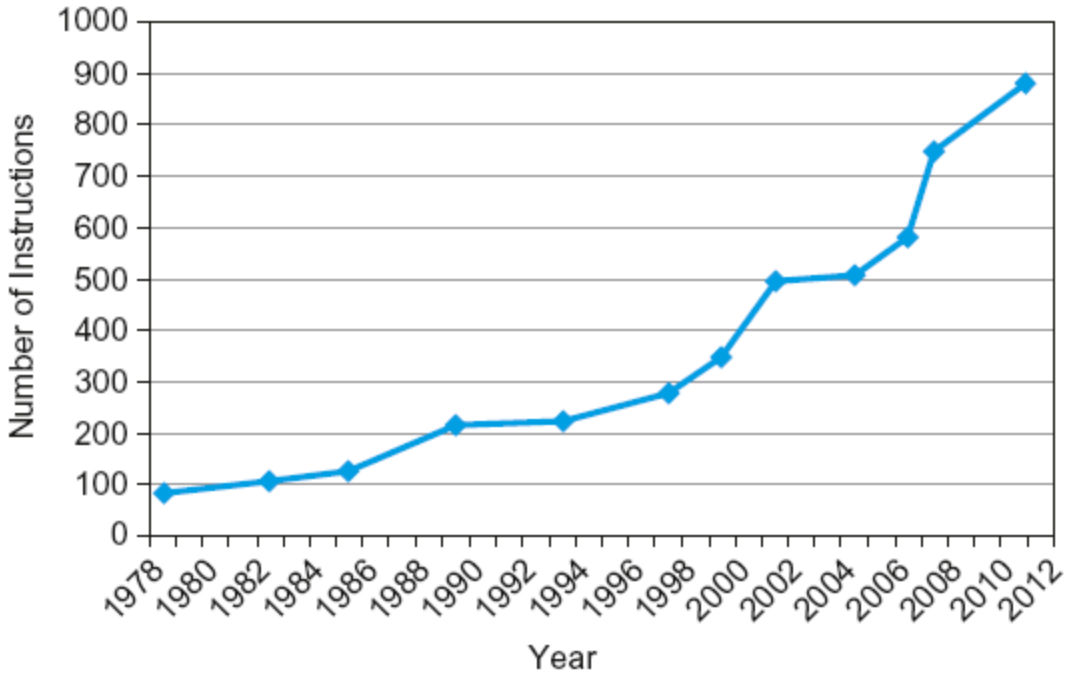
- Original debate
  - CISC proponents: easy for compiler, fewer code bytes
  - RISC proponents: better for optimizing compilers, can make run fast with simple chip design
- Current status
  - For desktop/server processors, choice of ISA not a technical issue
    - With enough hardware, can make anything run fast
    - Code compatibility more important
  - x86-64 adopted many RISC features
    - More register, use them for argument passing
    - Hardware translates instructions to simpler μops
  - For embedded processors, RISC makes sense: smaller, cheaper, less power

Fallacies

- Powerful instruction → higher performance
  - Fewer instructions required
  - But complex instructions are hard to implement
    - May slow down all instructions, including simple ones
  - Compilers are good at making fast code from simple instructions
- Use assembly code for high performance
  - But modern compilers are better at dealing with modern processors
  - More lines of code → more errors and less productivity

Fallacies (cont'd)

- Backward compatibility → instruction set doesn't change
  - But they do accrete more instructions



x86 instruction set

Pitfalls

- Sequential words are not at sequential addresses
  - Increment by 4 (32-bit) or 8 (64-bit), not by 1!
- Keeping a pointer to an automatic variable after procedure returns
  - e.g., passing pointer back via an argument
  - Pointer becomes invalid when stack popped